School of Future Tech

Case Study Report

on

Case Study 80: Public Transport Pass Management System

by

Dhruv Ketan Chavda

150096725060

Index

1. Introduction to the Case Study.
2. Problem Statement / Case Background (Abstract).
3. Problem Statement / Case Study Design.
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study.
5. Problem Statement / Case Study Implementation Details and Snapshots.
6. Problem Statement / Case Study Results and Conclusion.
7. References

1. Introduction to the Case Study

Public transportation systems are widely used in cities for daily commuting. Managing transport passes manually can lead to errors, inefficiencies, and difficulty in tracking pass validity and renewals.

This case study focuses on designing and implementing a **Public Transport Pass Management System using C++**. The system automates the process of issuing transport passes, tracking validity, searching pass details, and renewing passes through a menu-driven console application.

The project demonstrates how object-oriented programming concepts such as **classes, arrays, loops, conditional statements, and switch-case structures** can be used to build a practical transport management system.

2. Problem Statement / Case Background (Abstract)

⇒ Background

Traditional pass management systems often rely on manual registers, which makes it difficult to maintain records, track pass validity, and process renewals efficiently.

⇒ Abstract

This case study presents the design and implementation of a Transport Pass Management System using C++. The system allows users to:

- Apply for a new transport pass
- Check the validity of an existing pass
- Renew a pass when required

Each pass contains details such as pass ID, holder name, age, pass type, pass plan, validity days, and price. The system uses object-oriented programming to store and manage pass records efficiently. A menu-driven interface allows users to interact with the system until they choose to exit.

The program also applies a student discount and calculates additional charges when a pass is renewed.

3. Case Study Design

⇒ Data Storage

The system stores transport pass details using a **class structure called TransportPass**. Each object of this class represents a single transport pass.

The class stores the following attributes:

- **Pass ID** – Unique identifier for each transport pass
- **Holder Name** – Name of the pass holder
- **Age** – Age of the pass holder
- **Pass Type** – Indicates whether the pass holder is a Student or Regular passenger
- **Pass Plan** – Selected pass duration such as Monthly, Quarterly, Half-Yearly, or Yearly
- **Validity Days** – Number of days for which the pass remains valid
- **Price –** Cost of the pass depending on the selected plan

⇒ Control Flow Design

The system follows a **menu-driven control flow** that allows the user to perform different operations.

The program uses the following control structures:

- **do-while loop** – Repeats the menu until the user chooses to exit the program
- **switch-case statement** – Executes different operations based on the user's menu choice
- **if-else conditions** – Used for validation, applying student discounts, and checking pass validity

This structure ensures that the system runs continuously and allows users to interact with the program efficiently.

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study

➔ Programming Technology Used

Programming Language: **C++**
Compiler / Development Environment:

- Visual Studio Code

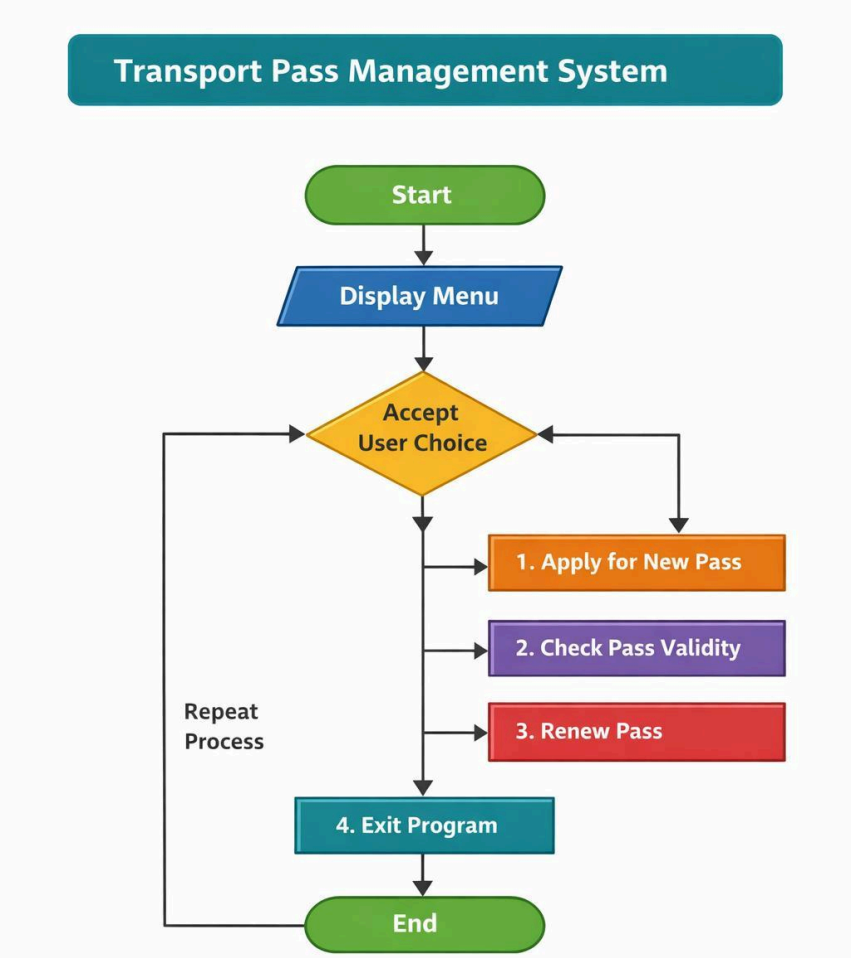
➔ Concepts Applied

The project uses the following programming concepts:

- Classes and Objects
- Arrays for storing multiple passes
- Loops (for loop, do-while loop)
- Conditional statements (if-else)
- Switch-case control structure
- Basic arithmetic operations for pricing and renewal calculations

➔ Algorithm

1. Start the program
2. Display the menu
3. Accept user choice
4. If choice = 1 →
   Apply for a new pass
5. If choice = 2 →
   Check Pass Validity
6. If choice = 3 →
   Renew Pass
7. Repeat until user selects Exit



5. Problem Statement / Case Study Implementation Details and Snapshots.

The **Transport Pass Management System** is implemented using C++ as a menu-driven console application. The system uses a class called **TransportPass** to store pass information such as pass ID, holder name, age, pass type, pass plan, validity days, and price.

Multiple passes are stored using an **array of objects**, which allows the system to manage several transport passes simultaneously.

```cpp
class TransportPass
{
private:
    int passID;
    string holderName;
    int age;
    string passType;
    string passPlan;
    int validityDays;
    float price;
```

⇒ Pass Application

The **applyForPass()** function allows users to create a new transport pass by entering the required details. The user selects the pass plan and the system assigns the corresponding validity days and price.

If the pass holder is a **student**, a 50% discount is applied to the pass price. The system displays the pass details and asks for confirmation before creating the pass.

```cpp
public:
    bool applyForPass()
    {
        int choice;

        cout << "Enter Pass ID: ";
        cin >> passID;

        cout << "Enter Holder Name: ";
        cin >> holderName;

        cout << "Enter Age: ";
        cin >> age;

        cout << "Is the pass holder a student? (1 = Yes, 0 = No): ";
        cin >> choice;

        if (choice == 1)
            passType = "Student";
        else
            passType = "Regular";

        cout << "\nSelect Pass Plan\n";
        cout << "1. Monthly\n";
        cout << "2. Quarterly\n";
        cout << "3. Half-Yearly\n";
        cout << "4. Yearly\n";
        cout << "Enter choice: ";
        cin >> choice;

        switch (choice)
        {
        case 1:
            passPlan = "Monthly";
            validityDays = 30;
            price = 500;
            break;

        case 2:
            passPlan = "Quarterly";
            validityDays = 90;
            price = 1400;
            break;
```

```cpp
        case 3:
            passPlan = "Half-Yearly";
            validityDays = 180;
            price = 2700;
            break;

        case 4:
            passPlan = "Yearly";
            validityDays = 365;
            price = 5000;
            break;

        default:
            cout << "Invalid plan\n";
            return false;
        }

        if (passType == "Student")
        {
            price *= 0.5;
            cout << "Student Discount Applied\n";
        }

        displayPassDetails();

        int confirm;

        cout << "\nConfirm Pass Application? (1 = Yes, 0 = No): ";
        cin >> confirm;

        if (confirm == 1)
        {
            cout << "Pass Created Successfully!\n";
            return true;
        }
        else
        {
            cout << "Pass Application Cancelled\n";
            return false;
        }
    }
```

⇒ Pass Validity Check

The **checkValidity()** function checks whether a pass is active or expired. The user enters the **Pass ID**, and the system searches for the corresponding pass in the array. If found, the program displays the pass details and validity status.

```
void checkValidity()
{
    if (validityDays > 0)
        cout << "Status: Active\n";
    else
        cout << "Status: Expired\n";
}
```

⇒ Pass Renewal

The **renewPass()** function allows the user to renew an existing pass. The user selects a renewal plan, and the system increases the validity days accordingly.

The renewal price is calculated and added to the existing pass price. Student discounts are applied if applicable. After renewal, the system displays the updated pass details.

```
void renewPass()
{
    int choice;

    cout << "\nSelect Renewal Plan\n";
    cout << "1. Monthly (30 days)\n";
    cout << "2. Quarterly (90 days)\n";
    cout << "3. Half-Yearly (180 days)\n";
    cout << "4. Yearly (365 days)\n";
    cout << "Enter choice: ";
    cin >> choice;

    float renewalPrice = 0;

    switch (choice)
    {
    case 1:
        validityDays += 30;
        renewalPrice = 500;
        break;

    case 2:
        validityDays += 90;
        renewalPrice = 1400;
        break;

    case 3:
        validityDays += 180;
        renewalPrice = 2700;
        break;

    case 4:
        validityDays += 365;
        renewalPrice = 5000;
        break;

    default:
        cout << "Invalid option\n";
        return;
    }
```

```
        if (passType == "Student")
        {
            renewalPrice *= 0.5;
        }

        price += renewalPrice;

        cout << "Pass renewed successfully!\n";
        cout << "Renewal Price Added: ₹" << renewalPrice << endl;
    }

    int getPassID()
    {
        return passID;
    }
};
```

6. Problem Statement / Case Study Results and Conclusion.

Findings

The developed Transport Pass Management System successfully performs the following operations:

- The system allows users to **apply for new transport passes** by entering passenger details and selecting a pass plan.
- The program automatically **calculates the price and validity period** based on the selected pass plan.
- A **student discount is applied automatically** when the pass holder is identified as a student.
- The system allows users to **check pass validity** by entering the Pass ID.
- The program allows **renewal of existing passes**, which increases the validity period and updates the total price.
- The menu-driven interface allows continuous interaction until the user chooses to exit.

Conclusion

This case study demonstrates the implementation of a **Transport Pass Management System using C++**. The system effectively manages transport pass records using object-oriented programming concepts such as classes, objects, arrays, loops, and conditional statements.

The program simplifies the process of issuing transport passes, checking pass validity, and renewing passes. The system provides a simple and efficient way to manage pass information through a menu-driven interface.

The project shows how basic C++ programming concepts can be applied to develop practical applications for transport management. Future improvements may include storing pass data in files, implementing real-time validity tracking using dates, and developing a graphical user interface.

7. References

- **C++ Programming Language Documentation** – https://www.cplusplus.com
- **ISO C++ Official Documentation** – https://en.cppreference.com
- **Visual Studio Code Documentation** – https://code.visualstudio.com/docs
- **Course Lecture Notes and Study Material** provided in the C++ Programming syllabus.